Unit 06

Python for Security

Module 1 — Technical Foundations · ~5 class periods

Curriculum by AJ Hammond — PNPT, CRTO, OSCP, BSCP

Why Python?

- **Readable** and beginner-friendly; **huge** library ecosystem.
- Used by real tools (Impacket) and most exploit proof-of-concepts.
- Cross-platform: write once, run on Windows, Linux, macOS.

> This builds directly on Bash week — same ideas (variables, loops, functions, conditionals), new language.

Learning objectives (1 of 2)

By the end of this unit you can:

- **Explain** why Python is popular in security; give two example tasks.
- **Run** Python two ways: interactive (`python3`) and a script (`python3 file.py`).
- **Create and use** variables of types `str`, `int`, `float`, `bool`, and convert between them.
- **Work with strings** (index, slice, `.split()`, f-strings) and build **lists** and **dictionaries**.

Learning objectives (2 of 2)

- **Write** `if`/`elif`/`else`, `for`, and `while`.
- **Define and call** a function with parameters and a return value.
- **Import** a module (`socket`) and read/write a file.
- **Use** `try`/`except` to handle errors instead of crashing.
- **Build** a working **TCP port scanner** for a single **isolated lab target**.
- **Apply** the rule: scanning runs **only** against the lab target.

Key vocabulary (1 of 2)

Term	Meaning
Interpreter	`python3` — runs Python code line by line.
Interactive (REPL)	A live `>>>` prompt; type a line, see the result.
Script	A `.py` file run top to bottom with `python3 file.py`.
Type	`str` text, `int` whole number, `float` decimal, `bool` True/False.
String	Text in quotes, e.g., `"22"` — text, not the number 22.
f-string	A string with values plugged in: `f"Port {p} open"`.
List	Ordered collection: `[22, 80, 443]`.
Dictionary	`key: value` pairs: `{22: "ssh"}`.

Key vocabulary (2 of 2)

Term	Meaning
Function	Reusable block defined with `def`, takes inputs, returns a result.
Module	Ready-made code you `import` (e.g., `socket`).
`socket`	Built-in module for making network connections.
Port	A numbered "door" where a service listens (22=SSH, 80=HTTP).
Timeout	Max time to wait before giving up, so it doesn't hang.
Exception	An error that happens while running.
`try` / `except`	Catch an exception and keep going instead of crashing.
Port scanner	A program that checks which ports on a host are open.

⚖️ Writing the tool yourself changes nothing

A port scanner is a real scanning tool — the language doesn't matter.

Building it in Python instead of running `nmap` does **not** make scanning an unauthorized host legal. The only legal target: the single isolated lab host you were assigned.

Ethics + scope check

- Port scanning without authorization can violate the **CFAA** and state law.
- **Before any network code**, confirm the host-only adapter:

```
ip addr        # expect a 192.168.56.x (or your lab) address
```

- Not sure you're on the right network or pointed at the right target? **Stop and ask.**

Discussion: In ~20 lines of Python you can check every port on a host. Why does building a tool *yourself* not change who you're allowed to point it at? What's the difference between *writing* a scanner and *running* it against a target?

Day 1

Why Python, and running it

Two ways to run Python

Interactive (REPL) — type a line, see the result instantly:

```
python3
>>> print("hello")
hello
>>> 2 + 2
4
>>> exit()
```

As a script — runs top to bottom:

```
python3 hello.py
```

The REPL is your sketchpad

```
python3
>>> ports = [22, 80]
>>> len(ports)
2
>>> ports.append(443)
>>> ports
[22, 80, 443]
```

- Try one line, see the result, build up your idea piece by piece.
- When a snippet works in the REPL, paste it into your script.

> Use the REPL to test small ideas before committing them to a file.

Python vs. Bash

Idea	Bash	Python
Variable	`name="Kali"`	`name = "Kali"` (spaces OK)
Use it	`$name`	`name` (no `$`)
Print	`echo "$name"`	`print(name)`
Comment	`# ...`	`# ...`
Blocks	`do` / `done`, `fi`	**indentation**

In Python, **indentation is syntax** — not just neatness. Wrong indentation is a real error.

String vs. number

```
print("22" + "1")     # 221  (text joined)
print(22 + 1)         # 23   (numbers added)
```

- `"22"` is **text** (a `str`); `22` is a **number** (an `int`).
- This matters: the `socket` module needs an **int** port, not a string.

Check your understanding (Day 1)

1. Name the two ways to run Python.
2. In Python, how do you mark a block of code (instead of `do`/`done`)?
3. Why is `"22"` different from `22`?

> Answer before the next slide.

Answers (Day 1)

1. **Interactive** (`python3` REPL) and **as a script** (`python3 file.py`).
2. By **indentation** — consistent spaces at the start of the lines.
3. `"22"` is text (`str`); `22` is a number (`int`). Math vs. joining behave differently.

Day 1 lab — banner script

```
#!/usr/bin/env python3
# hello.py — operator banner
operator = "Jordan Lee"     # put your name
target = input("Target IP for this session: ")
print("==== Port Scanner ====")
print(f"Operator: {operator}")
print(f"Target:   {target}")
```

```
python3 hello.py
```


Type a **lab** IP. An `IndentationError`? Check lines start at the left margin.

Exit ticket: Name the two ways to run Python and one difference.

Day 2

Variables, types, and strings

Curriculum by AJ Hammond — PNPT, CRTO, OSCP, BSCP

Types and converting

```
port = 22              # int
name = "ssh"           # str
ratio = 0.5            # float
is_open = True         # bool

type(port)             # <class 'int'>
int("22")              # convert text -> number  -> 22
str(22)                # convert number -> text  -> "22"
```

- `int(...)` and `str(...)` convert between text and numbers.

Worked example: convert types

```
port_text = "22"          # this is a string
port_text + 1             # ERROR: can't add int to str
int(port_text) + 1        # 23   (convert first)
```

- Input from the keyboard always arrives as a **string**.
- Convert with `int(...)` before doing math or scanning a port.

> The #1 scanner bug later: a port that's still a string. Remember `int()`.

Strings: index and slice

```
text = "192.168.56.10"
text[0]        # '1'   (first character)
text[0:3]      # '192' (a slice: positions 0,1,2)
```

- Indexing gets one character; slicing gets a piece.
- Counting starts at **0**.

Strings: `.split()` and f-strings

```
"22,80,443".split(",")     # ['22', '80', '443']  -> a list of strings
"  hi  ".strip()           # 'hi'  (trims spaces)

target = "192.168.56.10"
print(f"Scanning {target}")  # f-string plugs the variable in
```

- `.split()` is how you turn typed text into a list.
- An **f-string** (prefix `f`) plugs variables straight into the text.

Check your understanding (Day 2)

1. What does `int("80")` give you, and why use it?
2. What does `"22,80,443".split(",")` return?
3. Write an f-string that prints `Port 22 open` using a variable `p`.

> Predict, then check.

Answers (Day 2)

1. The number `80` (an `int`) — needed for math and for `socket`.
2. The list `['22', '80', '443']` — three **strings**.
3. `f"Port {p} open"` — the `{p}` is replaced by the variable's value.

Day 2 lab + exit ticket

- Extend `hello.py` to print an f-string summary using your variables.
- In the REPL: slice a string, split `"22,80,443"`, build an f-string status line.
- Journal each new piece of syntax.

Exit ticket: Write an f-string that prints `Scanning <target>` using a variable named `target`.

Day 3

Lists, dictionaries, conditionals, loops

Lists

```
ports = [21, 22, 80, 443]
ports[0]              # 21   (first item)
ports.append(8080)    # add to the end
len(ports)            # how many items
```

- An **ordered** collection in square brackets. Index from 0.

Worked example: loop a list

```
ports = [22, 80, 443]
for port in ports:
    print(f"Checking port {port}")
```

```
Checking port 22
Checking port 80
Checking port 443
```

- The loop runs once per item, with `port` taking each value in turn.

Dictionaries

```
services = {22: "ssh", 80: "http", 443: "https"}
services[80]                    # 'http'  (look up by key)
services.get(23, "unknown")     # 'unknown' if key missing
```

- A dictionary maps a **key** to a **value**.
- `.get(key, default)` avoids a crash when the key isn't there.

Why `.get()` beats `[ ]`

```
services = {22: "ssh", 80: "http"}
services[23]              # KeyError — crashes!
services.get(23, "?")     # '?'  — safe default
```

- `services[key]` **crashes** if the key is missing.
- `.get(key, default)` returns your fallback instead. Safer in a loop.

Conditionals: `if` / `elif` / `else`

```
if port == 22:
    print("This is SSH")
elif port == 80:
    print("This is HTTP")
else:
    print("Some other service")
```

- `==` tests equality. Note the **colon** and the **indented** body.

Loops

```
for port in ports:              # loop over a list
    print(f"Checking {port}")

n = 1
while n <= 3:                   # loop while a condition holds
    print(n)
    n = n + 1
```

A port scanner is really just: **loop over a list of ports and decide open/closed.**

Check your understanding (Day 3)

1. How do you add an item to the end of a list?
2. Given `{22:"ssh"}`, how do you safely look up port 23?
3. What punctuation must follow an `if` line, and what comes next?

> Reason it out first.

Answers (Day 3)

1. `mylist.append(item)`.
2. `services.get(23, "unknown")` — returns the default, no crash.
3. A **colon** (`:`), then an **indented** body on the next line(s).

Day 3 lab — build the data

```
#!/usr/bin/env python3
# scanner.py — simple TCP port scanner
# SAFETY: ISOLATED LAB TARGET ONLY.

import socket

ports = [21, 22, 23, 25, 53, 80, 110, 139, 143, 443, 445, 3306, 3389, 8080]

services = {
    21: "ftp", 22: "ssh", 23: "telnet", 25: "smtp", 53: "dns",
    80: "http", 110: "pop3", 139: "netbios", 143: "imap",
    443: "https", 445: "smb", 3306: "mysql", 3389: "rdp", 8080: "http-alt",
}
```

Day 4

Functions, the socket module, exceptions

Functions

```
def check_port(ip, port):
    return f"checking {ip}:{port}"

result = check_port("192.168.56.10", 22)   # call it
```

- `def` defines it; **parameters** (`ip`, `port`) are inputs.
- `return` hands a value back to whoever called it.

What is a port? An analogy

- A host is a **building**; each **port** is a numbered door.
- A service (SSH, web) listens behind a specific door (22, 80).
- A scan **knocks** on each door to see which ones answer.

> "Host up" (Bash week) = lights on. "Port open" = a specific door unlocked.

The `socket` module

```
import socket

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)   # a TCP socket
s.settimeout(0.5)                                       # give up after half a second
result = s.connect_ex((ip, port))  # returns 0 if the port is OPEN
s.close()
```

- `AF_INET` = IPv4, `SOCK_STREAM` = TCP.
- **`connect_ex` returns `0` when the port is open** (not `True`).
- **`settimeout`** stops it hanging on closed/filtered ports.

`connect_ex`: read the result

```
result = s.connect_ex((ip, port))
# result == 0   -> the door opened (OPEN)
# result != 0   -> no answer / refused (CLOSED)
return result == 0
```

- It returns a **number**, not `True`/`False`.
- `0` means success — so `result == 0` is your "is it open?" test.

> Expecting `True` here is a classic bug. `0` = open.

`try` / `except`

```
try:
    result = s.connect_ex((ip, port))
    return result == 0
except socket.error:
    return False
finally:
    s.close()
```

- `try`/`except` **catches** an error so one bad port doesn't crash the whole scan.
- `finally` always runs — here it closes the socket every time.

The `check_port` function (complete)

```
def check_port(ip, port):
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.settimeout(0.5)              # don't hang on closed ports
    try:
        result = s.connect_ex((ip, port))   # 0 means OPEN
        return result == 0
    except socket.error:
        return False
    finally:
        s.close()
```


Returns `True` (open) or `False` (closed/error). Test it on one known-open lab port.

Day 4 lab — the scan loop

```
target = input("Lab target IP to scan: ")
print(f"Scanning {target} (lab target only) ...")

open_ports = []
for port in ports:
    if check_port(target, port):
        name = services.get(port, "unknown")
        print(f"  [+] {port}/tcp open  ({name})")
        open_ports.append(port)

print(f"Done. {len(open_ports)} open port(s): {open_ports}")
```


Run with `python3 scanner.py` against the **lab target only**.

Day 5

Finish, harden & document the scanner

Save results to a file

```
with open("scan_results.txt", "w") as f:
    f.write(f"Target: {target}\n")
    f.write(f"Open ports: {open_ports}\n")
print("Saved to scan_results.txt")
```

```
cat scan_results.txt
```

- `open(..., "w")` opens a file for writing; `with` closes it automatically.

The complete `scanner.py`

```
#!/usr/bin/env python3
# scanner.py — simple TCP port scanner. SAFETY: LAB TARGET ONLY.
import socket

ports = [21, 22, 23, 25, 53, 80, 110, 139, 143, 443, 445, 3306, 3389, 8080]
services = {21:"ftp",22:"ssh",23:"telnet",25:"smtp",53:"dns",80:"http",
            110:"pop3",139:"netbios",143:"imap",443:"https",445:"smb",
            3306:"mysql",3389:"rdp",8080:"http-alt"}

def check_port(ip, port):
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.settimeout(0.5)
    try:
```

Check your results

- Compare your open-port list to the lab target's **known** open/closed ports.
- A Metasploitable-style target often shows 21, 22, 23, 25, 53, 80, 139, 445, 3306.
- A minimal Linux target maybe just 22 and 80. Closed ports correctly print nothing.

Common bugs: string vs int port, missing timeout (hangs), inverted `== 0`, bad indentation.

Spot the bug: string ports

```
ports = input("Ports: ").split(",")   # ['22','80'] (strings!)
for port in ports:
    check_port(target, port)           # socket needs an int
```

- Fix: convert each one — `check_port(target, int(port))`.
- `input()` and `.split()` always hand back **strings**.

Spot the bug: inverted test

```
if result != 0:           # BUG
    print("open")
```

- `connect_ex` returns `0` for **open**, so `!= 0` is backwards.
- Fix: `if result == 0: print("open")`. Otherwise closed ports look open.

Check your understanding (Day 5)

1. `connect_ex` returns `0` — does that mean open or closed?
2. Why does the scanner call `settimeout()`?
3. A port came from `input()`. What must you do before scanning it?

> Predict, then reveal.

Answers (Day 5)

1. **Open** — `0` means the connection succeeded.
2. So it gives up after a short wait instead of **hanging** on closed ports.
3. Convert it to a number with `int()` — `socket` needs an `int`.

Stretch goals

- Read the target from `sys.argv`: `python3 scanner.py 192.168.56.10`.
- Let the user enter a port **range** with `range()`.
- Time the scan with `time.time()`.
- **Banner grab:** `recv()` a few bytes from an open port; discuss what it reveals.
- Speed it up with `threading` (faster, but output can arrive out of order).
- Validate the target looks like an IP before scanning.

Lab deliverables

- Safety reminder in your own words + your assigned lab target IP.
- Working `hello.py` and `scanner.py` (pasted text).
- A screenshot of the scanner running.
- The **open-port list**, cross-checked against the target's known ports.
- The `scan_results.txt` contents.
- A **line-by-line annotation** of `scanner.py`.
- Reflection: one reliability feature + one place running it would be **illegal**.

Recap — what you can now do

- Run Python two ways; use types and convert between them.
- Strings (index/slice/split/f-strings), lists, dictionaries.
- `if` / `elif` / `else`, `for`, `while`.
- Functions with parameters + `return`.
- `import socket`, `settimeout`, `connect_ex`, `try` / `except`, file I/O.
- Built a real **TCP port scanner** — for the **lab target only**.

Quiz preview (assessment)

- Two ways to run Python?
- `print("22" + "1")` outputs what? (`221`)
- `services[80]` given `{22:"ssh",80:"http"}`? (`"http"`)
- `connect_ex` returns `0` when the port is…? (open)
- Spot the bug: `if result != 0: print("open")` — why is it backwards?
- Write a `check_port(ip, port)` function with timeout + `try`/`except`.

Discussion / exit ticket

"I *wrote* the port scanner myself, so I can test it on any website."

Correct that — in 2–3 sentences using the words *authorization* and *scope*.

Name one thing that makes your scanner reliable, and one place it would be **illegal** to run it.

Next up

Module 2: Reconnaissance — using these foundations to find and scan, *with permission*.

Curriculum by AJ Hammond — PNPT, CRTO, OSCP, BSCP

github.com/ajm4n · linkedin.com/in/aj-hammond

Curriculum by AJ Hammond — PNPT, CRTO, OSCP, BSCP